

‘Actions’ tab at the top. This section displays a list of workflow runs triggered by repository events. You should see a new workflow initiated by your recent push to the main branch. Click on the latest run to observe the workflow in progress. Inside, GitHub will display the sequence of jobs and steps defined in your `.yaml` file—such as checking out the code, building and pushing the Docker image, and deploying it to the server. Each step will have a status icon indicating success, failure, or if it’s currently in progress.

Once the workflow has completed successfully, verify that your application is deployed with the new changes. Open a browser and visit the server’s public IP address and application port (e.g., `http://<your-server-ip>:5000`). You should see your updated message—like ‘Hello from GitHub Actions!’—confirming the CD pipeline has deployed your changes (refer Figure 2). This step proves that every code push to main automatically triggers a build and deploy process, making your deployment seamless and consistent.

Enhancements and best practices for your CD workflow

Using environment variables for configuration

Environment variables play a crucial role in managing configuration data across different deployment environments such as development, staging, and production. Instead of hardcoding sensitive values like database credentials, API keys, or secret tokens into the source code, it is best practice to externalise them into environment variables. GitHub Actions supports this through encrypted secrets, which can be securely stored in your repository’s *Settings > Secrets* section. These secrets can then be referenced in your workflow files using the `${{ secrets.SECRET_NAME }}` syntax. This not only prevents accidental leakage

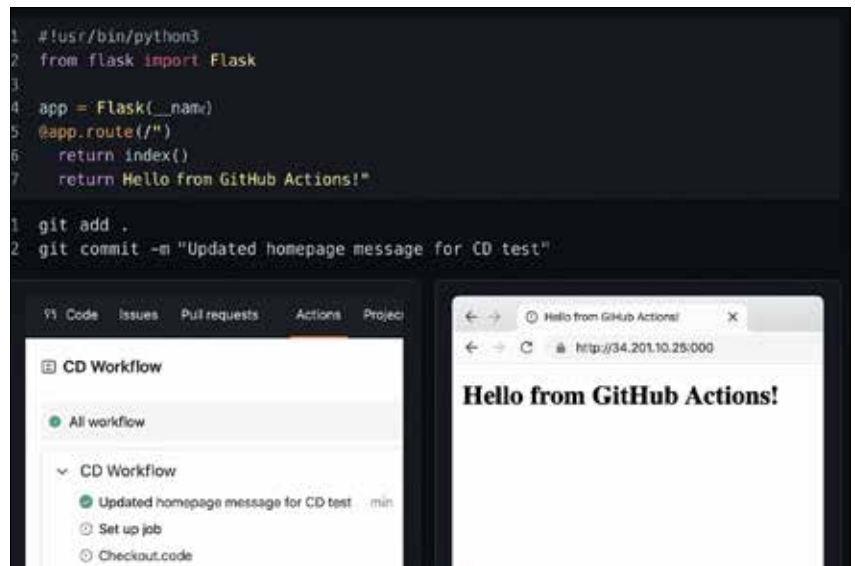


Figure 2: Continuous deployment workflow

of sensitive information but also enables seamless transitions between environments by updating only the environment-specific values without altering the codebase. Using tools like `.env` files in combination with Docker and `dotenv` libraries also makes it easy to load and manage configurations during both local development and production deployments.

Implementing health checks and rollbacks

To ensure that deployments are reliable and do not break production, implementing health checks is vital. Health checks are automated tests that verify if the application is running correctly after deployment. These can be simple HTTP GET requests to the root or `/health` endpoint of your application, checking for a successful status code (e.g., 200 OK). Many CD tools and orchestration platforms like Docker, Kubernetes, or GitHub Actions can integrate such checks into the pipeline. If a health check fails, an automatic rollback mechanism should be triggered. Rollbacks revert the application to the last known stable state, minimising downtime and user disruption. Implementing rollbacks

often involves keeping older Docker images or deployment configurations versioned and accessible, allowing quick recovery. This proactive approach to verifying and correcting deployments builds confidence in your CD pipeline and safeguards against critical failures in production.

Security considerations for your CD pipeline

Security is a top priority when designing and maintaining a CD pipeline. The pipeline must handle sensitive data like SSH keys, Docker Hub credentials, and deployment tokens—hence, it’s essential to store these securely using GitHub Actions secrets or a dedicated secrets manager like HashiCorp Vault. Always follow the principle of least privilege, granting each tool or script access only to the permissions it needs. Avoid running unnecessary scripts or installing untrusted packages in your workflow, and always validate inputs and dependencies to prevent supply chain attacks. Furthermore, it is good practice to regularly audit your CI/CD configuration for vulnerabilities, keep all dependencies up to date, and enable security scanning tools like